

如何搭配使用 Cloud Run 與 Gemini 函式呼叫功能

來源：<https://codelabs.developers.google.com/codelabs/how-to-cloud-run-gemini-function-calling?hl=zh-tw>

步驟數：10

如何將 Cloud Run 做為 Gemini 函式呼叫的端點。

目錄

1. [簡介](#)
2. [設定和需求](#)
3. [設定環境變數並啟用 API](#)
4. [建立服務帳戶來呼叫 Vertex AI](#)
5. [建立後端 Cloud Run 服務](#)
6. [建立前端服務](#)
7. [在本機執行 Frontend 服務](#)
8. [部署及測試 Frontend 服務](#)
9. [恭喜！](#)
10. [清除所用資源](#)

1. 簡介

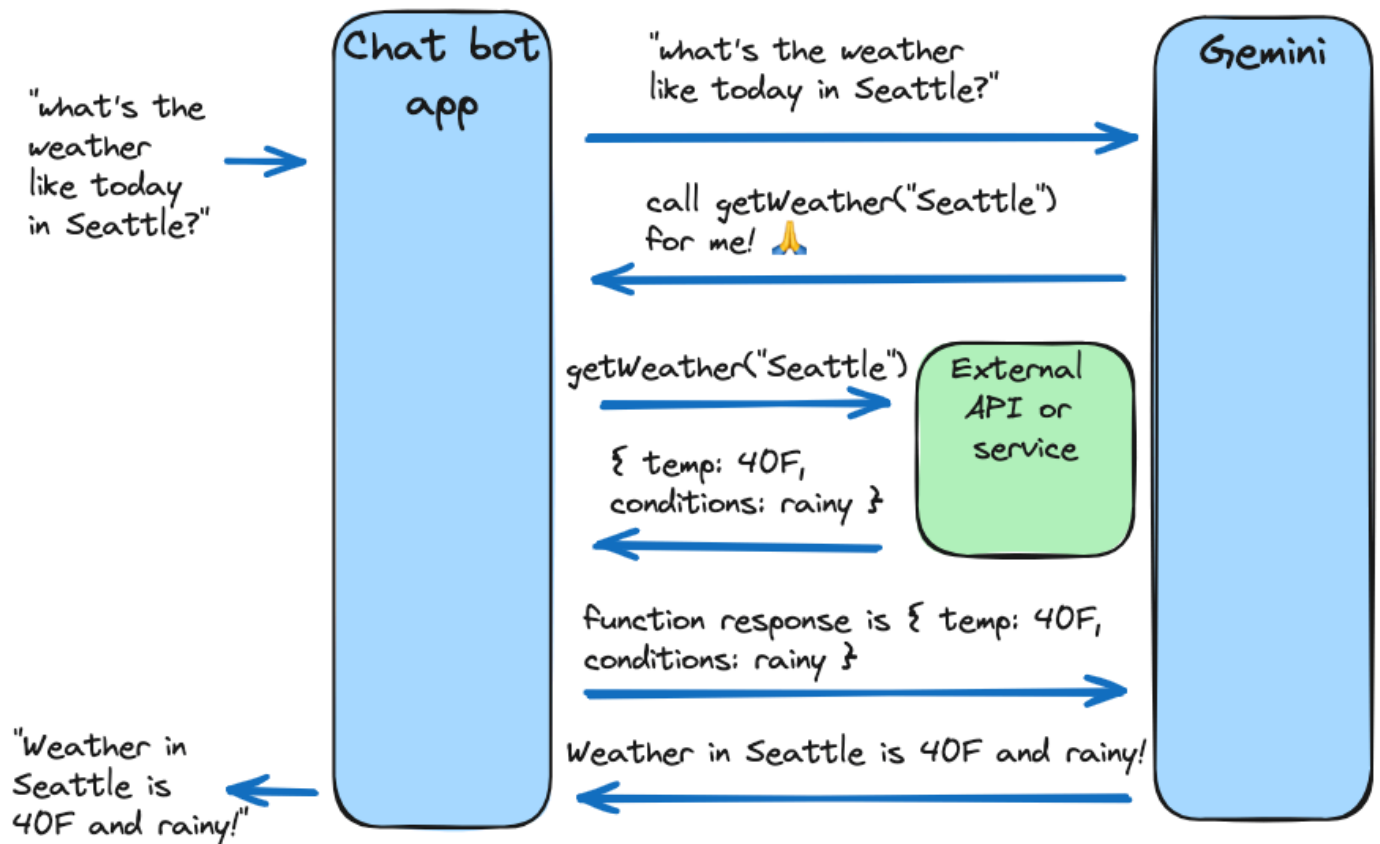
總覽

在本程式碼研究室中，您將瞭解如何使用名為「函式呼叫」的新功能，讓 Gemini 存取即時資料。如要模擬即時資料，請建構天氣服務端點，傳回 2 個地點的目前天氣。接著，您將建構由 Gemini 支援的即時通訊應用程式，並使用函式呼叫功能擷取目前的天氣資訊。

讓我們透過快速視覺化方式瞭解函式呼叫。

- 提示會要求特定地點的目前天氣位置
- 這個提示詞 + `getWeather()` 的函式合約會傳送給 Gemini
- Gemini 要求聊天機器人應用程式代表自己呼叫「`getWeather(Seattle)`」
- 應用程式會傳回結果 (華氏 40 度且下雨)
- Gemini 將結果傳回給呼叫端

再次提醒，Gemini 不會呼叫函式。開發人員必須呼叫函式，並將結果傳回 Gemini。



課程內容

- Gemini 函式呼叫的運作方式
- 如何將 Gemini 技術支援的聊天機器人應用程式部署為 Cloud Run 服務

步驟 2 · 約 0 分鐘

2. 設定和需求

必要條件

- 您已登入 Cloud Console。
- 您先前已部署第 2 代函式。舉例來說，您可以按照[部署 Cloud Functions 第 2 代快速入門導覽課程](#)操作。

啟用 Cloud Shell

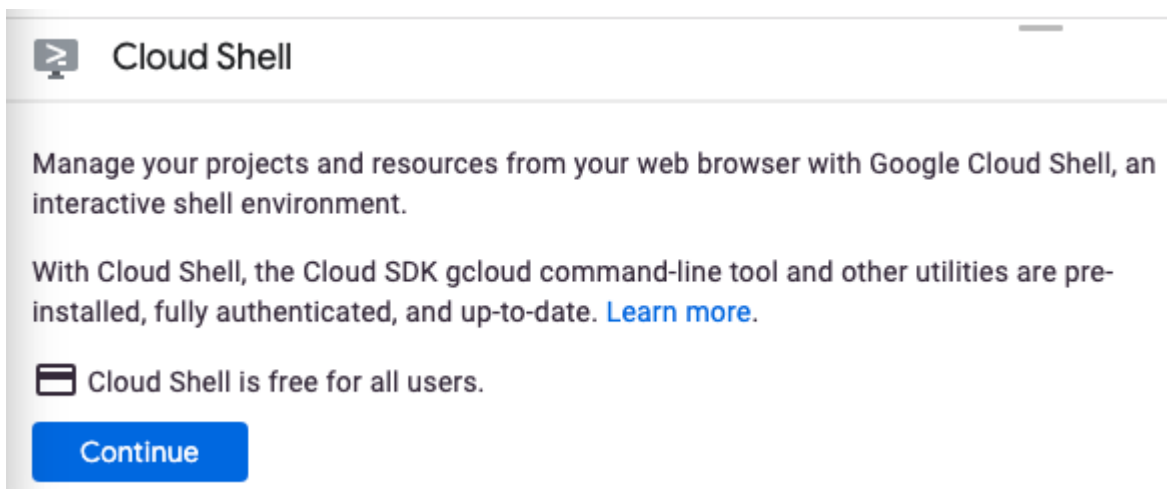
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



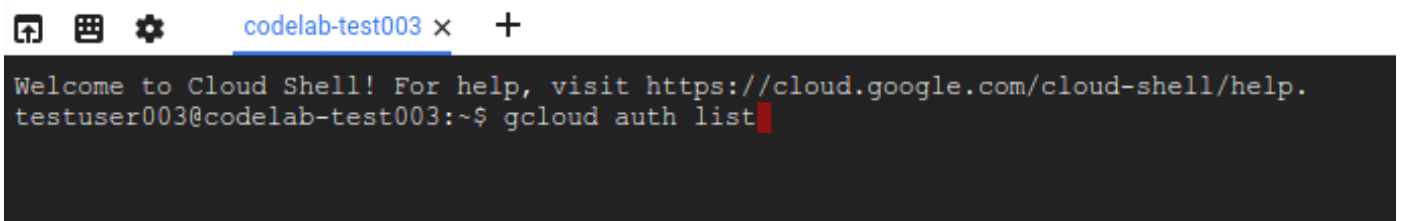
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

〇〇

```
gcloud auth list
```

指令輸出



```
Credentialed Accounts
ACTIVE  ACCOUNT
*      <my_account>@<my_domain.com>

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。第一次執行指令時，系統可能會提示您進行驗證。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意： 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

步驟 3 · 約 0 分鐘

3. 設定環境變數並啟用 API

設定環境變數

您可以設定本程式碼實驗室全程都會使用的環境變數。

```
PROJECT_ID=<YOUR_PROJECT_ID>
REGION=<YOUR_REGION, e.g. us-central1>
WEATHER_SERVICE=weatherservice
FRONTEND=frontend
SERVICE_ACCOUNT="vertex-ai-caller"
SERVICE_ACCOUNT_ADDRESS=$SERVICE_ACCOUNT@$PROJECT_ID.iam.gserviceaccount.com
```

啟用 API

開始進行本程式碼研究室之前，請先啟用數個 API。本程式碼研究室需要使用下列 API。執行下列指令即可啟用這些 API：

```
gcloud services enable run.googleapis.com \
  cloudbuild.googleapis.com \
  aiplatform.googleapis.com
```

步驟 4 · 約 0 分鐘

4. 建立服務帳戶來呼叫 Vertex AI

Cloud Run 會使用這個服務帳戶呼叫 Vertex AI Gemini API。

首先，請執行下列指令來建立服務帳戶：

```
gcloud iam service-accounts create $SERVICE_ACCOUNT \  
  --display-name="Cloud Run to access Vertex AI APIs"
```

其次，將 Vertex AI 使用者角色授予服務帳戶。

```
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \  
  --role=roles/aiplatform.user
```

步驟 5 · 約 0 分鐘

5. 建立後端 Cloud Run 服務

首先，請建立原始碼的目錄，然後 cd 到該目錄。

```
mkdir -p gemini-function-calling/weatherservice gemini-function-calling/frontend &&  
cd gemini-function-calling/weatherservice
```

接著，請建立 `package.json` 檔案，並加入以下內容：

```
{  
  "name": "weatherservice",  
  "version": "1.0.0",  
  "description": "",  
  "main": "app.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "dependencies": {  
    "express": "^4.18.3"  
  }  
}
```

接著，請使用下列內容建立 `app.js` 來源檔案。這個檔案包含服務的進入點，以及應用程式的主要邏輯。


```

const express = require("express");
const app = express();

app.get("/getweather", (req, res) => {
  const location = req.query.location;
  let temp, conditions;

  if (location == "New Orleans") {
    temp = 99;
    conditions = "hot and humid";
  } else if (location == "Seattle") {
    temp = 40;
    conditions = "rainy and overcast";
  } else {
    res.status(400).send("there is no data for the requested location");
  }

  res.json({
    weather: temp,
    location: location,
    conditions: conditions
  });
});

const port = parseInt(process.env.PORT) || 8080;
app.listen(port, () => {
  console.log(`weather service: listening on port ${port}`);
});

app.get("/", (req, res) => {
  res.send("welcome to hard-coded weather!");
});

```

部署天氣服務

您可以使用這個指令部署天氣服務。

```

gcloud run deploy $WEATHER_SERVICE \
  --source . \
  --region $REGION \
  --allow-unauthenticated

```

測試天氣服務

您可以使用 curl 驗證這 2 個地點的天氣：

```
WEATHER_SERVICE_URL=$(gcloud run services describe $WEATHER_SERVICE \
    --platform managed \
    --region=$REGION \
    --format='value(status.url)')
```

```
curl $WEATHER_SERVICE_URL/getweather?location=Seattle
```

```
curl $WEATHER_SERVICE_URL/getweather?location\=New%20Orleans
```

西雅圖氣溫為華氏 40 度，下著雨；紐奧良氣溫為華氏 99 度，而且總是潮濕。

步驟 6 · 約 0 分鐘

6. 建立前端服務

首先，請使用 `cd` 指令進入前端目錄。

```
cd gemini-function-calling/frontend
```

接著，請建立 `package.json` 檔案，並加入以下內容：

```
{
  "name": "demo1",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node app.js",
    "nodemon": "nodemon app.js",
    "cssdev": "npx tailwindcss -i ./input.css -o ./public/output.css --watch",
    "tailwind": "npx tailwindcss -i ./input.css -o ./public/output.css",
    "dev": "npm run tailwind && npm run nodemon"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "@google-cloud/vertexai": "^0.4.0",
    "axios": "^1.6.7",
    "express": "^4.18.2",
    "express-ws": "^5.0.2",
    "htmx.org": "^1.9.10"
  },
  "devDependencies": {
    "nodemon": "^3.1.0",
    "tailwindcss": "^3.4.1"
  }
}
```

接著，請使用下列內容建立 `app.js` 來源檔案。這個檔案包含服務的進入點，以及應用程式的主要邏輯。

注意：為簡化本程式碼研究室，本程式碼研究室的即時通訊記錄為暫時性。記錄只會保留給這個特定 Cloud Run 服務例項。如要在 Cloud Run 服務的多個執行個體中保留即時通訊記錄，請參閱[這項程式碼研究室](#)，瞭解如何使用工作階段中介軟體將即時通訊記錄固定至工作階段 ID。

```

const express = require("express");
const app = express();
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
const path = require("path");

const fs = require("fs");
const util = require("util");
const { spinnerSvg } = require("./spinnerSvg.js");

const expressWs = require("express-ws")(app);

app.use(express.static("public"));

const {
  VertexAI,
  FunctionDeclarationSchemaType
} = require("@google-cloud/vertexai");

// get project and location from metadata service
const metadataService = require("./metadataService.js");

// instance of Gemini model
let generativeModel;

// 1: define the function
const functionDeclarations = [
  {
    function_declarations: [
      {
        name: "getweather",
        description: "get weather for a given location",
        parameters: {
          type: FunctionDeclarationSchemaType.OBJECT,
          properties: {
            location: {
              type: FunctionDeclarationSchemaType.STRING
            },
            degrees: {
              type: FunctionDeclarationSchemaType.NUMBER,
              "description":
                "current temperature in fahrenheit"
            },
            conditions: {
              type: FunctionDeclarationSchemaType.STRING,
              "description":
                "how the weather feels subjectively"
            }
          }
        }
      }
    ]
  }
]

```

```

        },
        required: ["location"]
    }
}
]
}
];

// on instance startup
const port = parseInt(process.env.PORT) || 8080;
app.listen(port, async () => {
    console.log(`demo1: listening on port ${port}`);

    const project = await metadataService.getProjectId();
    const location = await metadataService.getRegion();

    // Vertex client library instance
    const vertex_ai = new VertexAI({
        project: project,
        location: location
    });

    // Instantiate models
    generativeModel = vertex_ai.getGenerativeModel({
        model: "gemini-1.0-pro-001"
    });
});

const axios = require("axios");
const baseUrl = "https://weatherservice-k6msmyp47q-uc.a.run.app";

app.ws("/sendMessage", async function (ws, req) {

    // this chat history will be pinned to the current
    // Cloud Run instance. Consider using Firestore &
    // Firebase anonymous auth instead.

    // start ephemeral chat session with Gemini
    const chatWithModel = generativeModel.startChat({
        tools: functionDeclarations
    });

    ws.on("message", async function (message) {
        let questionToAsk = JSON.parse(message).message;
        console.log("WebSocket message: " + questionToAsk);

        ws.send(`<div hx-swap-oob="beforeend:#toupdate"><div
            id="questionToAsk"

```

```

                class="text-black m-2 text-right border p-2 rounded-lg ml-
24">
                ${questionToAsk}
            </div></div>`);

        // to simulate a natural pause in conversation
        await sleep(500);

        // get timestamp for div to replace
        const now = "fromGemini" + Date.now();

        ws.send(`<div hx-swap-oob="beforeend:#toupdate"><div
                id=${now}
                class=" text-blue-400 m-2 text-left border p-2 rounded-lg mr-
24">
                ${spinnerSvg}
            </div></div>`);

        const results = await chatWithModel.sendMessage(questionToAsk);

        // Function calling demo
        let response1 = await results.response;
        let data = response1.candidates[0].content.parts[0];

        let methodToCall = data.functionCall;
        if (methodToCall === undefined) {
            console.log("Gemini says: ", data.text);
            ws.send(`<div
                id=${now}
                hx-swap-oob="true"
                hx-swap="outerHTML"
                class="text-blue-400 m-2 text-left border p-2 rounded-lg mr-
24">
                ${data.text}
            </div>`);

            // bail out - Gemini doesn't want to return a function
            return;
        }

        // otherwise Gemini wants to call a function
        console.log(
            "Gemini wants to call: " +
            methodToCall.name +
            " with args: " +
            util.inspect(methodToCall.args, {
                showHidden: false,
                depth: null,

```

```

        colors: true
    })
};

// make the external call
let jsonReturned;
try {
    const responseFunctionCalling = await axios.get(
        baseUrl + "/" + methodToCall.name,

        {
            params: {
                location: methodToCall.args.location
            }
        }
    );
    jsonReturned = responseFunctionCalling.data;
} catch (ex) {
    // in case an invalid location was provided
    jsonReturned = ex.response.data;
}

console.log("jsonReturned: ", jsonReturned);

// tell the model what function we just called
const functionResponseParts = [
    {
        functionResponse: {
            name: methodToCall.name,
            response: {
                name: methodToCall.name,
                content: { jsonReturned }
            }
        }
    }
];

// // Send a follow up message with a FunctionResponse
const result2 = await chatWithModel.sendMessage(
    functionResponseParts
);

// This should include a text response from the model using the response
content
// provided above
const response2 = await result2.response;
let answer = response2.candidates[0].content.parts[0].text;
console.log("answer: ", answer);

```

```

        ws.send(`<div
                                id=${now}
                                hx-swap-oob="true"
                                hx-swap="outerHTML"
                                class="text-blue-400 m-2 text-left border p-2 rounded-lg mr-
24">
                                ${answer}
                                </div>`);
    });

    ws.on("close", () => {
        console.log("WebSocket was closed");
    });
});

function sleep(ms) {
    return new Promise((resolve) => {
        setTimeout(resolve, ms);
    });
}

```

為 tailwindCSS 建立 `input.css` 檔案。

```

@tailwind base;
@tailwind components;
@tailwind utilities;

```

建立 tailwindCSS 的 `tailwind.config.js` 檔案。

```

/** @type {import('tailwindcss').Config} */
module.exports = {
  content: ["./**/*.{html,js}"],
  theme: {
    extend: {}
  },
  plugins: []
};

```

建立 `metadataService.js` 檔案，取得已部署 Cloud Run 服務的專案 ID 和區域。這些值會用於例項化 Vertex AI 用戶端程式庫的執行個體。

注意：如要在本機執行應用程式，您必須在下列第 1 和第 2 行設定專案 ID 和區域：


```

const your_project_id = "YOUR_PROJECT_ID";
const your_region = "YOUR_REGION";

const axios = require("axios");

module.exports = {
  getProjectId: async () => {
    let project = "";
    try {
      // Fetch the token to make a GCF to GCF call
      const response = await axios.get(
        "http://metadata.google.internal/computeMetadata/v1/project/project-
id",
        {
          headers: {
            "Metadata-Flavor": "Google"
          }
        }
      );

      if (response.data == "") {
        // running locally on Cloud Shell
        project = your_project_id;
      } else {
        // use project id from metadata service
        project = response.data;
      }
    } catch (ex) {
      // running locally on local terminal
      project = your_project_id;
    }

    return project;
  },

  getRegion: async () => {
    let region = "";
    try {
      // Fetch the token to make a GCF to GCF call
      const response = await axios.get(
        "http://metadata.google.internal/computeMetadata/v1/instance/region",
        {
          headers: {
            "Metadata-Flavor": "Google"
          }
        }
      );
    }
  }
};

```

```

    if (response.data == "") {
      // running locally on Cloud Shell
      region = your_region;
    } else {
      // use region from metadata service
      let regionFull = response.data;
      const index = regionFull.lastIndexOf("/");
      region = regionFull.substring(index + 1);
    }
  } catch (ex) {
    // running locally on local terminal
    region = your_region;
  }
  return region;
}
};

```

建立名為 `spinnerSvg.js` 的檔案

```

module.exports.spinnerSvg = `

```

建立新的 `public` 目錄。

```

mkdir public
cd public

```

現在要為前端建立 `index.html` 檔案，其中會使用 `htmx`。

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta
      name="viewport"
      content="width=device-width, initial-scale=1.0"
    />
    <script
      src="https://unpkg.com/htmx.org@1.9.10"
      integrity="sha384-
D1Kt99CQMDuVetoL1lrYwg5t+9QdHe7NLX/SoJYkXDFfx37iInKRy5xLSi8nO7UC"
      crossorigin="anonymous"
    ></script>

    <link href="./output.css" rel="stylesheet" />
    <script src="https://unpkg.com/htmx.org/dist/ext/ws.js"></script>

    <title>Demo 2</title>
  </head>
  <body>
    <div id="herewego" text-center>
      <!-- <div id="replaceme2" hx-swap-oob="true">Hello world</div> -->
      <div
        class="container mx-auto mt-8 text-center max-w-screen-lg"
      >
        <div
          class="overflow-y-scroll bg-white p-2 border h-[500px] space-y-4
rounded-lg m-auto"
        >
          <div id="toupdate"></div>
        </div>
        <form
          hx-trigger="submit, keyup[keyCode==13] from:body"
          hx-ext="ws"
          ws-connect="/sendMessage"
          ws-send=""
          hx-on="htmx:wsAfterSend: document.getElementById('message').value
= ''"
        >
          <div class="mb-6 mt-6 flex gap-4">
            <textarea
              rows="2"
              type="text"
              id="message"
              name="message"
              class="block grow rounded-lg border p-6 resize-none"
              required

```

```
        >
What&apos;s is the current weather in Seattle?</textarea
        >
        <button
            type="submit"
            class="bg-blue-500 text-white px-4 py-2 rounded-lg text-
center text-sm font-medium"
        >
            Send
        </button>
    </div>
</form>
</div>
</div>
</body>
</html>
```

步驟 7 · 約 0 分鐘

7. 在本機執行 Frontend 服務

首先，請確認您位於 Codelab 的 `frontend` 目錄。

```
cd .. && pwd
```

接著，執行下列指令來安裝依附元件：

```
npm install
```

在本機執行時使用 ADC

如果您在 Cloud Shell 中執行，表示您已在 Google Compute Engine 虛擬機器上執行。應用程式預設憑證會自動使用與這個虛擬機器相關聯的憑證 (如執行 `gcloud auth list` 所示)，因此不必使用 `gcloud auth application-default login` 指令。您可以跳到「在本機執行應用程式」一節

不過，如果您是在本機終端機上執行 (即不在 Cloud Shell 中)，則需要使用應用程式預設憑證向 Google API 進行驗證。您可以選擇 1) 使用憑證登入 (前提是您同時具備 Vertex AI 使用者和資料儲存庫使用者角色)，或 2) 模擬本程式碼實驗室中使用的服務帳戶登入。

選項 1) 使用 ADC 的憑證

如要使用憑證，請先執行 `gcloud auth list`，確認您在 `gcloud` 中的驗證方式。接著，您可能需要授予身分「Vertex AI 使用者」角色。如果您的身分具備「擁有者」角色，您就已擁有這個 Vertex AI 使用者角色。如果沒有，可以執行這項指令，將 Vertex AI 使用者角色和 Datastore 使用者角色授予您的身分。

```
USER=<YOUR_PRINCIPAL_EMAIL>

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member user:$USER \
  --role=roles/aiplatform.user

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member user:$USER \
  --role=roles/datastore.user
```

然後執行下列指令

```
gcloud auth application-default login
```

選項 2) 模擬 ADC 的服務帳戶

如要使用本程式碼研究室建立的服務帳戶，您的使用者帳戶必須具備[服務帳戶權杖建立者角色](#)。您可以執行下列指令來取得這個角色：

```
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member user:$USER \  
  --role=roles/iam.serviceAccountTokenCreator
```

接著，執行下列指令，搭配服務帳戶使用 ADC

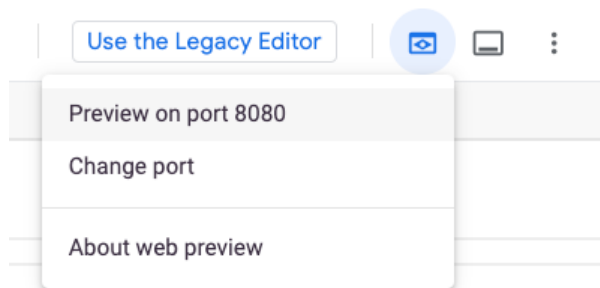
```
gcloud auth application-default login --impersonate-service-  
account=$SERVICE_ACCOUNT_ADDRESS
```

在本機執行應用程式

最後，您可以執行下列指令碼來啟動應用程式。這個開發指令碼也會從 `tailwindCSS` 產生 `output.css` 檔案。

```
npm run dev
```

開啟「網頁預覽」按鈕，然後選取「預覽通訊埠 8080」，即可預覽網站



步驟 8 · 約 0 分鐘

8. 部署及測試 Frontend 服務

首先，請執行下列指令啟動部署作業，並指定要使用的服務帳戶。如果未指定服務帳戶，即會使用預設的運算服務帳戶。

```
gcloud run deploy $FRONTEND \  
  --service-account $SERVICE_ACCOUNT_ADDRESS \  
  --source . \  
  --region $REGION \  
  --allow-unauthenticated
```

在瀏覽器中開啟前端的服務網址。提出問題「西雅圖目前的天氣如何？」，Gemini 應會回答「目前氣溫為 40 度，正在下雨。」如果你問「波士頓目前的天氣如何？」，Gemini 會回覆「我無法完成這項要求。可用的天氣 API 沒有波士頓的資料。」

步驟 9 · 約 0 分鐘

9. 恭喜！

恭喜您完成本程式碼研究室！

建議您參閱 [Cloud Run](#)、[Vertex AI Gemini API](#) 和 [函式呼叫](#) 的說明文件。

涵蓋內容

- Gemini 函式呼叫的運作方式
 - 如何將 Gemini 技術支援的聊天機器人應用程式部署為 Cloud Run 服務
-

10. 清除所用資源

為避免產生意外費用 (例如，如果這個 Cloud Run 服務的叫用次數不慎超過[免費層級的每月 Cloud Run 叫用次數配額](#))，您可以刪除 Cloud Run 服務，或刪除在步驟 2 中建立的專案。

如要刪除 Cloud Run 服務，請前往 Cloud Run Cloud 控制台 (<https://console.cloud.google.com/functions/>)，然後刪除您在本程式碼實驗室中建立的 \$WEATHER_SERVICE 和 \$FRONTEND 服務。

您也可以刪除 `vertex-ai-caller` 服務帳戶或撤銷 Vertex AI 使用者角色，避免無意間呼叫 Gemini。

如要刪除整個專案，請前往 <https://console.cloud.google.com/cloud-resource-manager>，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
